AWS DevOps & CI/CD

Learning Notes

1. Introduction to AWS DevOps

DevOps is a combination of Development (Dev) and Operations (Ops). It is a software development approach that brings developers and IT operations teams together to build, test, and deploy applications faster and more reliably.

DevOps focuses on automation, collaboration, continuous integration, continuous delivery, and monitoring.

Objectives of DevOps

- Improve software development speed.
- Automate repetitive tasks.
- Reduce manual errors.
- Improve collaboration between development and operations teams.
- Deliver software updates frequently.
- Maintain application reliability.

DevOps Lifecycle

The DevOps lifecycle consists of the following stages:

- **Plan:** Define project requirements and tasks.
- **Code:** Developers write application code.
- **Build:** Convert source code into a deployable application.
- **Test:** Check the application for errors.
- **Release:** Prepare the application for deployment.
- **Deploy:** Make the application available to users.
- **Operate:** Maintain the deployed application.
- **Monitor:** Track application performance and identify issues.

2. Git and GitHub Fundamentals

What is Git?

Git is a distributed version control system used to track changes in source code. It allows developers to manage different versions of a project and collaborate with other developers.

Features of Git

- Tracks changes in files.
- Maintains project history.

- Supports multiple development branches.
- Allows developers to work together.
- Helps recover previous versions.
- Supports offline development.

What is GitHub?

GitHub is a cloud-based platform used to host Git repositories. It helps developers store, manage, share, and collaborate on projects.

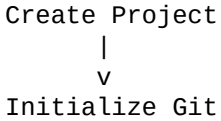
Difference Between Git and GitHub

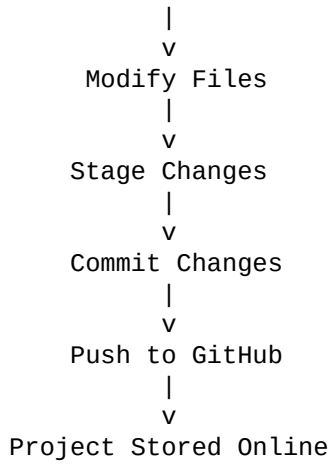
Git	GitHub
Version control system	Cloud-based code hosting platform
Installed on a computer	Accessed through a website or tools
Tracks code changes	Stores and shares Git repositories
Works offline	Requires internet for remote collaboration
Developed by Linus Torvalds	Owned by Microsoft

Important Git Commands

Command	Description
`git --version`	Checks Git installation
`git init`	Initializes a repository
`git status`	Displays file changes
`git add .`	Stages all changed files
`git commit -m "message"`	Saves changes in Git history
`git log`	Displays commit history
`git branch`	Lists branches
`git checkout`	Switches branches
`git clone`	Copies a remote repository
`git push`	Uploads commits to a remote repository
`git pull`	Downloads and integrates remote changes

Basic Git Workflow





3. Understanding Version Control

Definition

Version control is a system that records changes made to files over time. It allows developers to track, compare, and restore different versions of their projects.

Types of Version Control Systems

1. Local Version Control

- Stores file versions on a local computer.
- Useful for individual projects.
- Limited collaboration capabilities.

2. Centralized Version Control

- Stores project versions on a central server.
- Developers access the same central repository.
- Examples include SVN and CVS.

3. Distributed Version Control

- Every developer has a complete copy of the repository.
- Supports offline work.
- Makes collaboration easier.
- Git is an example.

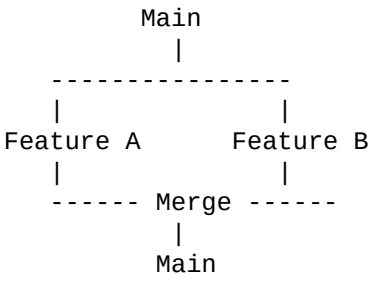
Advantages of Version Control

- Tracks project history.
- Prevents accidental loss of code.
- Supports teamwork.
- Helps identify errors.
- Allows recovery of previous versions.

- Supports parallel development.

Git Branching

A branch is a separate line of development within a repository. For example:



Branches allow developers to work on new features without directly changing the main version of the application.

4. Continuous Integration and Continuous Deployment

What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Delivery or Continuous Deployment. It is a software development practice that automates the process of building, testing, and delivering applications.

Continuous Integration (CI)

Continuous Integration is the practice of frequently combining code changes from different developers into a shared repository. Whenever code is updated, automated processes can:

- Detect code changes.
- Build the application.
- Run automated tests.
- Identify errors.
- Provide feedback to developers.

Example: A developer pushes updated code to GitHub. An automated workflow starts and checks whether the application builds successfully.

Continuous Delivery (CD)

Continuous Delivery ensures that application code is automatically built, tested, and prepared for release. The final production deployment generally requires manual approval.

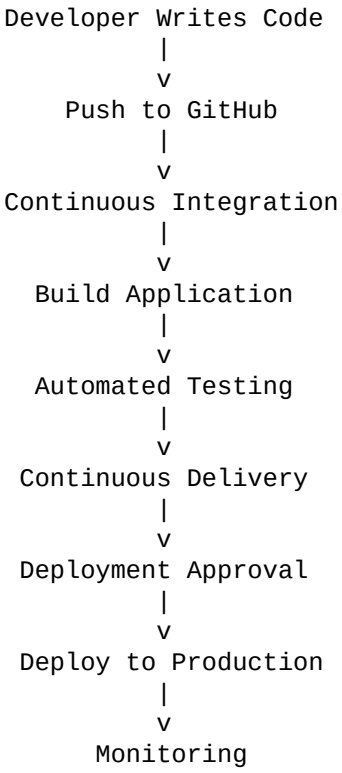
Continuous Deployment (CD)

Continuous Deployment automatically releases successfully tested code into the production environment without requiring manual approval at each release.

Difference Between CI, Continuous Delivery and Continuous Deployment

Feature	Continuous Integration	Continuous Delivery	Continuous Deployment
Main purpose	Integrate code changes	Prepare code for release	Automatically release code
Build process	Automated	Automated	Automated
Testing	Automated	Automated	Automated
Production release	Not included	Usually requires approval	Automatically performed
Main benefit	Early error detection	Reliable releases	Faster software delivery

CI/CD Workflow



In continuous deployment, successful testing can trigger production deployment automatically.

5. AWS CI/CD Concepts and Services

Amazon Web Services provides several services that help automate software development and deployment.

Important AWS DevOps Services

1. AWS CodeCommit

AWS CodeCommit was a managed Git repository service. It was used to store and manage source code securely. However, AWS discontinued availability to new customers in 2024, so GitHub or another supported Git provider is a practical alternative for new projects.

2. AWS CodeBuild

AWS CodeBuild is a fully managed build service. Functions:

- Compiles source code.
- Runs automated tests.
- Produces build artifacts.
- Scales according to workload.

3. AWS CodeDeploy

AWS CodeDeploy automates application deployments to supported compute environments. Functions:

- Automates application deployment.
- Supports deployment to Amazon EC2 and other supported targets.
- Helps manage application updates.
- Supports deployment strategies such as rolling and blue/green deployments, depending on the target platform.

4. AWS CodePipeline

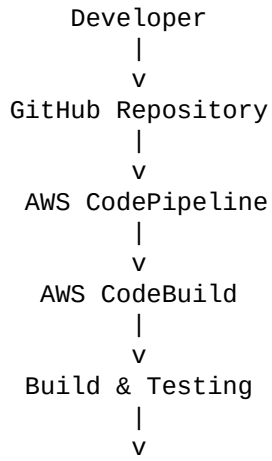
AWS CodePipeline is a continuous delivery service that automates software release workflows. Functions:

- Connects source, build, test, and deployment stages.
- Automates release workflows.
- Integrates with other AWS services.
- Tracks pipeline execution.

5. Amazon EC2

Amazon EC2 provides virtual servers in the cloud. Applications can be deployed and hosted on EC2 instances.

AWS CI/CD Architecture



```
AWS CodeDeploy
      |
      v
  Amazon EC2
      |
      v
Deployed Application
```

Advantages of AWS CI/CD

- Reduces manual deployment work.
- Automates software delivery.
- Improves deployment consistency.
- Helps detect errors early.
- Supports scalable cloud infrastructure.
- Integrates with multiple development tools.

6. Creating a Basic Deployment Pipeline

Aim

To understand and create a basic CI/CD deployment pipeline using GitHub and AWS services.

Requirements

- AWS account.
- Git installation.
- GitHub account.
- Basic HTML website.
- AWS CodePipeline.
- AWS CodeBuild.
- AWS CodeDeploy.
- Amazon EC2 instance (for an EC2-based deployment).

Step 1: Create a GitHub Repository

1. Open GitHub.
2. Create a new repository.
3. Give it a suitable name, such as `aws-cicd-project`.
4. Add your website files.
5. Commit and push your code.

Step 2: Prepare the AWS Environment

1. Sign in to the AWS Management Console.
2. Open the AWS CodePipeline service.

3 Select Create Pipeline.

4 Enter a pipeline name.

5 Configure the required service permissions and connections.

Step 3: Configure the Source Stage

1 Select GitHub as the source provider.

2 Connect your GitHub account.

3 Select the repository.

4 Select the branch.

5 Configure the source trigger.

Whenever a new commit is pushed, the pipeline can detect the change and start execution.

Step 4: Configure the Build Stage

1 Open AWS CodeBuild.

2 Create a build project.

3 Connect the GitHub source.

4 Select the required build environment.

5 Configure the build instructions.

6 Run the build process.

A simple `buildspec.yml` example for an HTML website:

```
version: 0.2

phases:
  build:
    commands:
      - echo "Building the website"

artifacts:
  files:
    - '**/*'
```

This example packages the project files as build artifacts. It does not compile or test application code.

Step 5: Configure the Deployment Stage

For an EC2-based deployment:

1 Prepare an EC2 instance.

2 Install and configure the required web server.

3 Prepare an `appspec.yml` deployment file.

4 Create an AWS CodeDeploy application.

5 Create a deployment group.

6 Connect the deployment stage to CodePipeline.

Step 6: Test the Pipeline

1 Make a small change to the website.

2 Commit the updated code.

3 Push the changes to GitHub.

4 Open AWS CodePipeline.

5 Observe the pipeline execution.

6 Check whether the build and deployment stages succeed.

7 Open the website to verify the deployed update.

Expected Result: The updated application is deployed automatically after the configured pipeline stages complete successfully.

7. Documenting the Deployment Workflow

Documentation explains the complete deployment process, configuration, tools, and results.

Important Documentation Sections

1. Project Title

AWS DevOps and CI/CD Pipeline Implementation.

2. Objective

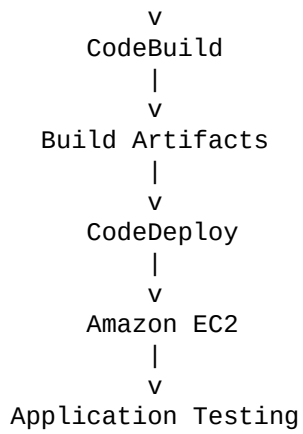
To create an automated deployment pipeline using GitHub and AWS services.

3. Tools Used

- Git
- GitHub
- AWS CodePipeline
- AWS CodeBuild
- AWS CodeDeploy
- Amazon EC2

4. Workflow

```
Source Code Development
          |
          v
      Git Commit
          |
          v
    GitHub Push
          |
          v
    CodePipeline
          |
```



5. Testing and Verification

The pipeline should be tested by updating the source code and checking whether the changes successfully reach the deployment environment.

6. Screenshots to Include

- GitHub repository.
- Git commit history.
- AWS CodePipeline configuration.
- CodeBuild execution result.
- CodeDeploy deployment status.
- Final deployed website.

7. Result

A basic CI/CD deployment workflow was configured to demonstrate how source code can be integrated, built, tested, and deployed using GitHub and AWS services.

8. Advantages of AWS DevOps and CI/CD

- **Faster Development:** Automation reduces the time required for software delivery.
- **Improved Collaboration:** Git and GitHub support teamwork.
- **Early Error Detection:** Automated testing identifies issues during development.
- **Reliable Deployment:** Standardized pipelines reduce manual mistakes.
- **Scalability:** AWS services support applications with changing workloads.
- **Better Productivity:** Developers can focus more on application development.
- **Continuous Improvement:** Frequent feedback helps improve software quality.

9. Real-World Applications

AWS DevOps and CI/CD are commonly used in:

- E-commerce application deployment.
- Banking and financial software.
- Mobile application backend deployment.
- Cloud-based web hosting.
- Enterprise software development.
- Microservices deployment.
- Automated software updates.

10. Skills Gained

After completing this module, the following skills and knowledge can be gained:

Area	Skills Learned
Git	Repository management and Git commands
GitHub	Source code hosting and collaboration
Version Control	Tracking and managing code changes
CI/CD	Understanding automated software delivery
AWS	Understanding AWS DevOps services
Pipeline	Creating a basic deployment workflow
Documentation	Recording deployment steps and results

11. Conclusion

AWS DevOps and CI/CD play an important role in modern software development. Git and GitHub help developers manage source code and track changes, while CI/CD automates the building, testing, and deployment process. AWS services such as CodePipeline, CodeBuild, and CodeDeploy help create automated deployment workflows.

By understanding these concepts and creating a basic deployment pipeline, developers can improve software delivery, reduce manual work, and build reliable cloud-based applications.

Quick Revision – Important Terms

Term	Meaning
DevOps	Development + Operations
Git	Distributed version control system
GitHub	Git repository hosting platform
CI	Continuous Integration

Term	Meaning
CD	Continuous Delivery / Continuous Deployment
Pipeline	Automated sequence of development and deployment stages
CodeBuild	AWS managed build service
CodeDeploy	AWS automated deployment service
CodePipeline	AWS continuous delivery orchestration service
EC2	Elastic Compute Cloud

Note: These are learning notes covering all six objectives. The pipeline example is a practical outline; actual deployment requires configuring AWS permissions, service connections, build instructions, and deployment files.